



Option B - Handwritten Number Recognition System (HNRS)

Project Report

Prepared by:

Anh Luu – 103979983

Alexandre Henry Edouard Rollet – 105963412

Jaime Amadeu Sousa – 105963137

Oscar Lorenzo Gilbert Bonifay – 105963108

Unit Name & Code:

Intelligent Systems (COS30018)

Due Date:

02/10/2025

Table of Contents

Introduction	3
Overall System Architecture	4
Data Preprocessing	5
Image Segmentation	6
Handwritten Digit Recognition.....	6
Overall System Functions and Performance	7
Critical Analysis of the Implementation	7
Conclusion.....	8

Introduction

This report has the purpose of explaining the code we created to answer problem B given in the Intelligent system unit. This unit is dedicated to learning how automated systems and machine learning work and leads us to developing various skills related to these topics. We had the choice between 4 projects, and we decided to commit to option B.

Option B focuses on developing a Handwritten Number Recognition System (HNRS) as a group project for 3-4 students. This project involves creating a system to recognize handwritten numbers, broken into subproblems like image acquisition, digit localization, individual digit recognition, and multi-digit number construction. The HNRS utilizes ML techniques and libraries like numpy or PyTorch with a minimum requirement to train models on the MNIST dataset and provide predictions. System requirements include options for image acquisition from folders or files, and a methodology covering image preprocessing, segmentation, ML model training and evaluation.

Overall System Architecture

Our Handwritten Number Recognition System is built in a modular way, meaning each part of the program has a specific role. We separated the code into parts that handle the interface, image processing, and machine learning. This helps make the system easier to understand, test, and improve.

The system is made of four main parts:

1. Graphical User Interface:

The interface was created with CustomTkinter, which is a modern version of Tkinter. It makes the program look cleaner and allows features like dark mode.

From the interface, the user can:

1. Upload an image from their computer
2. Take a new picture using the webcam
3. See messages showing what the program is doing (like training progress or defection steps)
4. View the final result showing the detected and recognized digits

2. Image Preprocessing:

This step happens in the `process_image()` function

It prepares the input image before recognition. The program turns the image into grayscale, resizes it, and uses adaptive thresholding to separate the digits from the background. Then, some filters and cleaning operations are applied to remove noise. The final result is a black and white image that's easier to analyze in the next step.

3. Segmentation:

In this part, the program finds where each digit is in the image. It uses OpenCV's `findContours()` function to detect shapes that look like digits. Each detected shape is surrounded by a rectangle (called a bounding box) and sorted from left to right. These cropped digit images are then sent to the recognition part.

4. Recognition and Machine Learning:

For recognizing the digits, we used a Convolutional Neural Network (CNN) built with PyTorch.

The model is trained on the MNIST dataset. If no trained model is found, the program automatically starts the training process and saves it as `model.pt`

Once ready, the model predicts each digit one by one and shows the result directly in the interface.

In summary, each part of the system works together smoothly: the interface handles the interaction with the user, the preprocessing and segmentation prepare the image, and the CNN performs the recognition.

Data Preprocessing

Before the system can recognize any handwritten digits, the images must go through a series of transformations to make them easier to analyze. The data preprocessing has a very important role in the project because if the input images are too noisy or unclear, the model will have a hard time detecting and recognizing the numbers correctly. Good preprocessing helps improve both accuracy and speed during the recognition.

In our project, the preprocessing stage is fully handled by the `process_image()` function, which uses the OpenCV library (`cv2`). This function takes the image selected by the user (or captured by the webcam) and applies several operations one after another to make the image cleaner and more consistent.

Here is a detailed explanation of each step:

1. Grayscale conversion:

The first thing the program does is convert the image to grayscale using `cv2.imread(..., cv2.IMREAD_GRAYSCALE)`. Since the goal is to detect the shape of the digits, color information is not necessary. By working only with brightness levels (from black to white), we reduce the amount of data to process, which makes the following steps faster and more efficient.

2. Resizing the image:

The image is then resized to a fixed size of 600×600 pixels. This ensures that all images have similar proportions, which helps the rest of the pipeline work consistently. Resizing is also useful for keeping the processing speed stable, especially when the user uploads large images from their computer or webcam.

3. Noise reduction with Gaussian blur:

Handwritten digits often contain imperfections or small marks that can confuse the algorithm. To reduce these, we use a Gaussian blur filter (`cv2.GaussianBlur()`). This operation smooths the image by slightly blending nearby pixels. As a result, the digits become cleaner, and small unwanted dots or marks are removed.

4. Adaptive thresholding:

After blurring, the program applies adaptive Gaussian thresholding (`cv2.adaptiveThreshold()`), which turns the grayscale image into a black-and-white (binary) image. This means every pixel is either completely black (background) or completely white (part of a digit).

Unlike a simple fixed threshold, the adaptive version calculates the threshold value for each region of the image separately. This is very useful when the lighting is uneven, for example if one side of the image is darker than the other. This method helps ensure that the digits are always well separated from the background.

5. Morphological cleaning:

Even after thresholding, small pieces of noise or gaps can remain in the image. To deal with that, the program applies morphological operations such as opening (which removes small white noise) and dilation (which strengthens the white parts of the digits). These operations are performed using small rectangular kernels that slide over the image and adjust pixel patterns. The goal is to make the digits thicker, more continuous, and easier to detect later.

After all these steps, the final image is much cleaner, with clear white digits on a black background. This processed image is now ready to be passed to the next part of the pipeline: image segmentation.

By applying these transformations, we make sure the digits stand out from the background, reduce the chances of detecting false shapes, and prepare a solid foundation for the recognition model. In short, this preprocessing phase improves the quality of the input data, which directly improves the reliability of the overall system!

Image Segmentation

After the image has been preprocessed and cleaned, the next stage of the HNRS concerns the image segmentation. The purpose of this part is to locate and isolate each handwritten digit from the image so that they can be recognized and treated separately by the neural network.

1. Purpose:

The individual treatment allows accurate recognition of the image. This makes the system focus on one digit at a time, reducing confusion and ensuring that numbers appearing close together are properly distinguished and analyzed in sequence.

2. Explanation:

The segmentation process begins with detecting the shapes present in the binary image produced during preprocessing. Using shape detection methods (with the contour function in code), the system identifies the contours or outlines of objects that resemble digits. Each detected contour is then surrounded by a bounding rectangle, which defines the area containing a potential digit.

Once all possible digit areas are detected, the system applies several filtering criteria to remove unwanted shapes. Very small regions, which usually represent noise, and very large regions corresponding to the background/overlapping shapes, are discarded. The system also checks the proportions of each detected region to ensure that only shapes with realistic digit-like dimensions are kept (that concerns the aspect-ratio function in code).

After filtering, the valid bounding boxes are sorted from left to right. This ordering ensures that multi-digit numbers are read and recognized in the correct sequence. The segmented digits are then visually outlined on the image using colored rectangles, providing a clear visual confirmation of successful detection. The segmentation process results in a set of clearly defined regions, each containing one digit ready for recognition.

3. Summary:

In summary, the image segmentation component serves as the bridge between preprocessing and recognition. Through contour detection, bounding box extraction, and careful filtering, the system accurately identifies and separates handwritten digits from the input image. This structured approach allows the Handwritten Number Recognition System to handle multi-digit inputs effectively.

Handwritten Digit Recognition

The most crucial part of this system is the hand recognition system. This component is responsible for identifying which number each segment represents. It uses a Convolutional Neural Network (CNN) built with the help of the pytorch library to perform this task accurately.

1. Purpose:

The recognition stage converts the visual patterns of handwritten digits into meaningful numerical values. By analyzing the shape, structure, and features of each segmented digit, the neural network predicts the most probable number it represents. This process allows the system to interpret handwriting automatically.

2. CNN Architecture:

The recognition model is implemented as a CNN. It is a type of deep learning model well-suited for image classification. The architecture consists of two convolutional layers followed by pooling, dropout, and fully connected layers.

- The first convolutional layer extracts simple features such as edges and curves from the input image.
- The second convolutional layer identifies more complex features like loops or intersections that distinguish one digit from another.
- Max pooling is used to reduce the size of the feature maps, keeping only the most important information and improving efficiency.
- Dropout layers help prevent overfitting by randomly disabling some neurons during training, ensuring that the model generalizes well to unseen data.
- Finally, two fully connected layers transform the extracted features into a probability distribution over the ten possible digits (0 to 9).

The model outputs a numerical prediction corresponding to the most likely digit.

3. Model Training:

CNN is trained using the MNIST dataset, a collection of 70000 handwritten digit images. Each image has the particularity of being 28x28 pixels and labeled with the correct digit. During training, the model learns to associate pixel patterns with specific numbers by adjusting its internal weights.

The training process involves multiple epochs, where the model repeatedly passes through the training data. Each batch of images is fed forward through the network, and the difference between the prediction and the actual label is calculated using a cross-entropy

loss function. The Adam optimizer then updates the model's weights to minimize this loss. Once training is complete, the model's parameters are saved locally so that the system can reuse the trained model in future runs without retraining.

4. Digit Recognition Process:

Before prediction, each digit is resized to 28×28 pixels (the same size as MNIST images) and normalized to ensure consistency with the training data. The model then performs inference, outputting a probability for each possible digit from 0 to 9. The digit with the highest probability is selected as the final prediction.

This process is repeated for all segmented digits, and the results are combined to form the final recognized number. The recognized digits are displayed in the program interface, and a message confirming the detected number is added to the technical log. This allows users to visually verify both the recognition output and the processing sequence.

The CNN model demonstrates strong accuracy on handwritten digits due to its layered feature extraction and effective training on the MNIST dataset. The automatic retraining option ensures the system remains functional even if the model file is missing.

The integration of PyTorch for model handling and OpenCV for preprocessing creates a robust and reliable recognition pipeline that can adapt to various handwriting styles and image conditions.

Overall System Functions and Performance

Import and Libraries

Let's dive into the collection of tools and libraries we harnessed to bring this project to life. We've got:

- 'os' to manage file paths and directories,
- 'customtkinter' for a stylish, tailored interface build on top of Tkinter,
- 'tkinter' with 'filedialog' to let users select files,
- 'datetime' to stamp our messages with the current time,
- 'PIL' (Pillow) to adjust images, such as resizing and prepping them for the interface,
- 'numpy' as 'np' to handle number crunching, especially with arrays,
- 'cv2' from OpenCV to tackle vision tasks like fine-tuning image thresholds and detecting contours.

Then we venture into PyTorch territory with:

- 'torch' and 'torch.nn' to construct neural networks,
- 'optim' to polish them during training,
- 'DataLoader' and 'TensorDataset' to organize data chunks, along with 'datasets' and 'transforms' from torchvision to fetch the MNIST dataset,
- 'threading' to keep training running smoothly in the background without locking up the interface.

Loading MNIST Data Function

The 'load_mnist' function is quite simple to follow. It's all about getting the MNIST dataset ready to teach a digit recognizer. It sets up a process to turn images into tensors and adjusts them using MNIST's average and spread values to help the model learn better. It grabs the training dataset, mixes it up, and organizes it into batches of 64 using a DataLoader, while also pulling in the test dataset. The function then hands back the training batches, the total number of samples, and the test dataset. We'll call this a standard PyTorch way to prepare data; using DataLoader makes it easier to loop through batches during training.

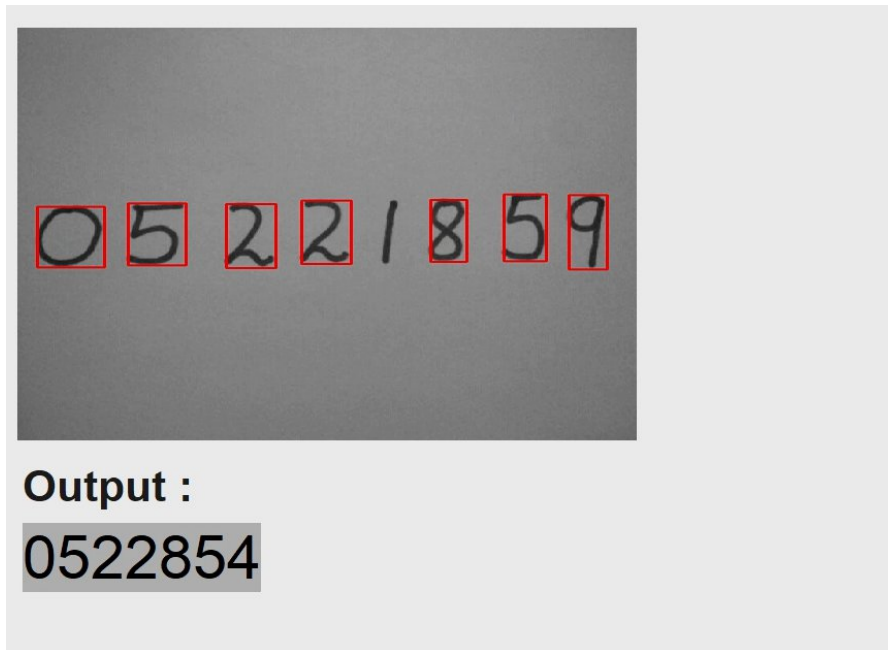
GUI and Input Method

The user is able to insert any modern image format into our HNRS system (both image files and webcam output is acceptable, less if the image is either corrupted or in the wrong format).

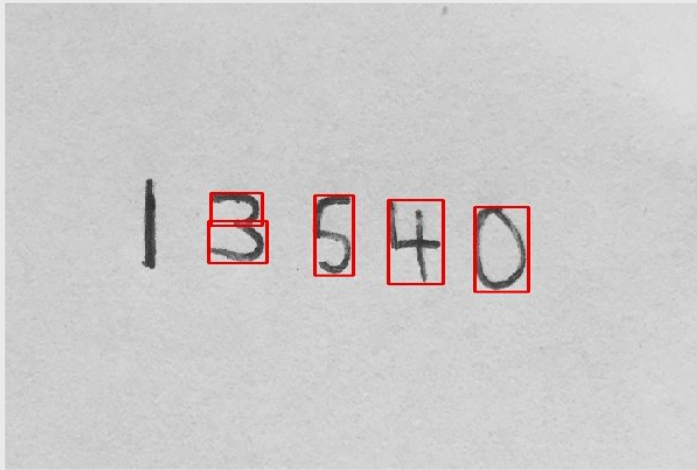
System Performance

When the user opts to use the application without our pre-trained model named 'model.pt', the application itself will try to fetch that model and use what has been pretrained. If no model is found, then the application automatically trains one and uses it afterwards. In the code, it can be specified how many epochs are used to train the model (it takes about 20 minutes to train a model if none is found).

Critical Analysis of the Implementation



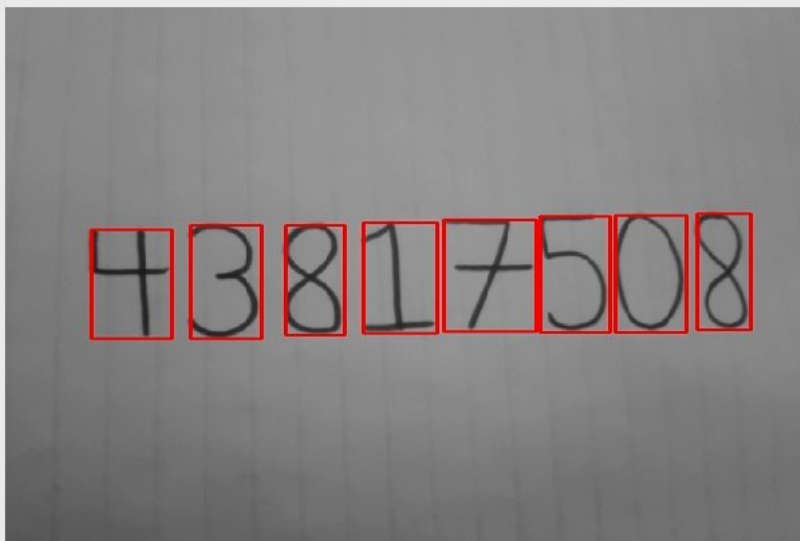
05221859 – Digit 1 written like an l does not get detected, 9 is confused for 4.



Output :

32546

13540 – Digit 1 written like an l does not get detected, 3 is confused as 2 objects (3 & 2), 0 is confused as 6.



Output :

73828768

43817508 – Digits 4s and 7s are confused, 1s and 2s are confused, 5s and 6s are confused, and the final 7 & 8 are confused.



Output :

353452806

351452806 – Digit 1 are confused as 3.

Conclusion

The project we developed successfully implemented a functional handwritten number recognition system using machine learning techniques that we studied during the 12 weeks of tutorial classes and the GUI we made helps any user, proficient or not in coding, be able to interact with application that we tried to make very user friendly. Comprehensive testing with the MNIST dataset and many handwritten numbers we tried demonstrated reliable accuracy and performance in most use cases. Overall, this project really taught us how to use and implement a machine learning component into a coding application.